PSA解析

担当者	陽輝 飯山陽輝
Purpose	
Object	

raw ファイルの読み方 (psa_decode.py)

電荷の計算方法（psa_decode.py）

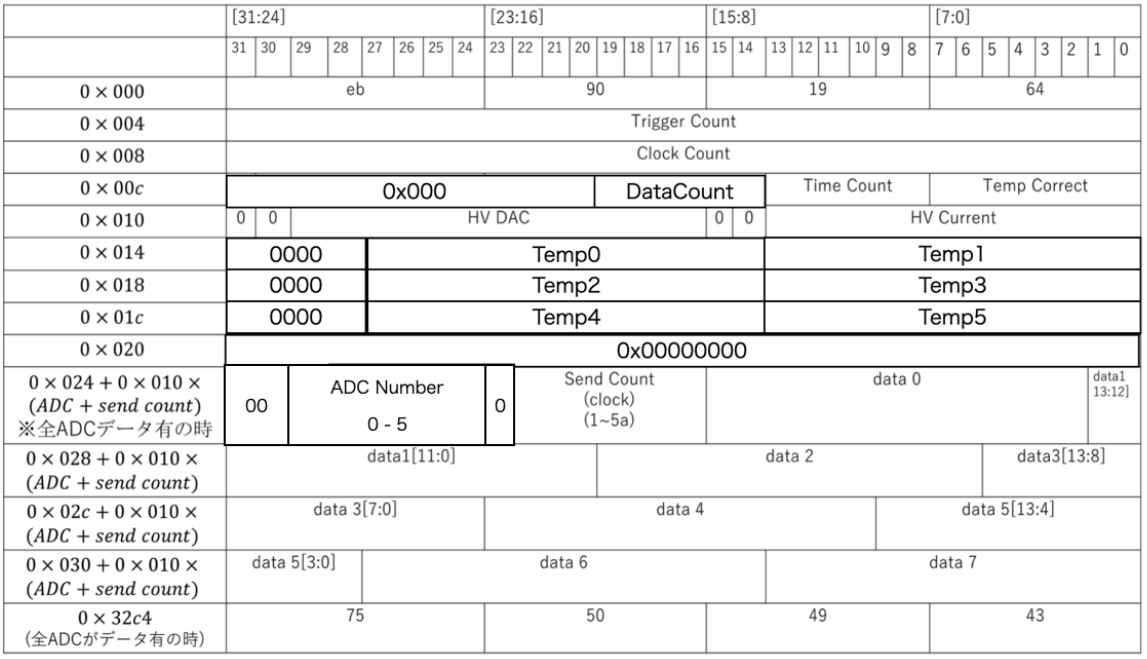
- PSA_decode
  - イベント
  - ADCデータ
  - 波形解析
  - 通常のHDF5出力
  - `-adc` 指定時
  - 実行例

バタフライマップから実座標への変換ファイル

- peak_search.py
  - 入力
  - イベント選別
  - ピーク検出
  - 出力
  - 入力ディレクトリ
  - 注意点
- grid_spect.py
  - 入力
  - イベント選別
  - 位置補正とピクセル分割
  - HGスペクトル
  - 位置マップ

raw ファイルの読み方 (psa_decode.py)

以下のフォーマットに従ってデータが入力されている



	[31:24]								[23:16]								[15:8]								[7:0]							
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0×000	eb								90								19								64							
0×004	Trigger Count																															
0×008	Clock Count																															
$0\times00c$	0x000																DataCount								Time Count							
0×010	0	0	HV DAC														0	0	HV Current													
0×014	0000								Temp0																Temp1							
0×018	0000								Temp2																Temp3							
$0\times01c$	0000								Temp4																Temp5							
0×020	0x00000000																															
$0\times024+0\times010\times(ADC+send\ count)$ ※全ADCデータ有の時	00		ADC Number 0 - 5					0	Send Count (clock) (1~5a)								data 0														data1 13:12]	
$0\times028+0\times010\times(ADC+send\ count)$	data1[11:0]																data 2								data3[13:8]							
$0\times02c+0\times010\times(ADC+send\ count)$	data 3[7:0]																data 4												data 5[13:4]			
$0\times030+0\times010\times(ADC+send\ count)$	data 5[3:0]								data 6																data 7							
$0\times32c4$ (全ADCがデータ有の時)	75								50								49								43							

ADCデータは、Send CountごとにADC numberの降順で格納される。Send Count 1なら

```
0b100000 → 0b010000 → … → 0b000001
```

の順に並び、その後、同じ並びをSend Count 2、Send Count 3.....と繰り返す。

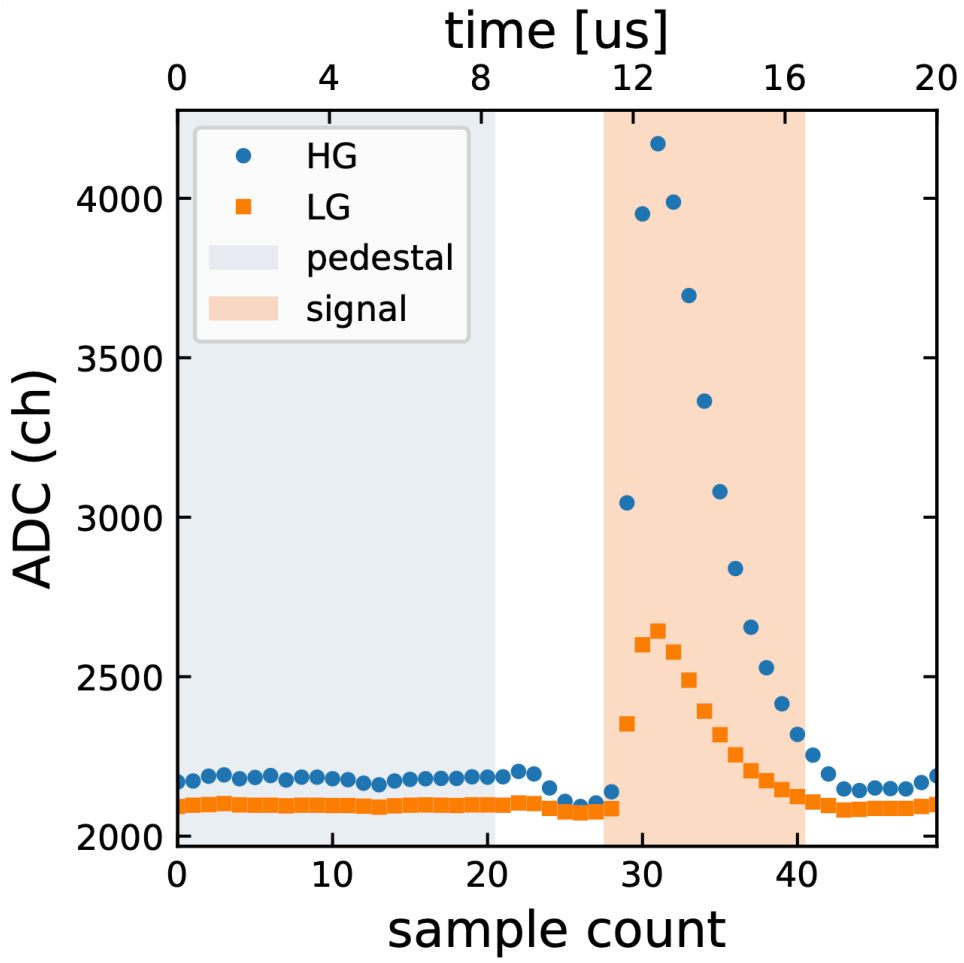
ADCの読み出し順序はそのADCデータがなくなる。例えば ADC 1と2 の時ならば

- ADC number 0b000010 の Send Count 1
- ADC number 0b000001 の Send Count 1
- ADC number 0b000010 の Send Count 2
- ADC number 0b000001 の Send Count 2
- ・・・・

の順で詰まる。

電荷の計算方法（psa_decode.py）

読み出した波形でデータからADCの最大値を取得、ピーク前4点、ピーク後9点のADCを台形積分して計算



現状ではパイルアップの処理は決まっていない。

→ pedestal値とpeak位置でカット、などで対応

PSA_decode

`psa_decode.py` は、PSAの `.dat` ファイルを解析し、イベント情報・ADC波形・温度などをHDF5（`.h5`）に変換する。

イベント

- ヘッダー：`EB 90 19 64`
- フッター：`75 50 49 43`

- ヘッダーからフッターまでを1イベントとして読み込む。
- トリガー番号、クロック値、6チャンネルのヒット情報、6点の温度ADC値を取得する。
- 時刻は先頭イベントを0秒とし、次式で計算する。

```
event_time = (clock - first_clock) / 1,000,000
```

温度の順番は `TL, TR, ML, MR, BL, BR`。

ADCデータ

ADCデータは16バイト単位で、各ブロックに以下を含む。

```
ADC番号、サンプリング番号、HG0～HG3、LG0～LG3
```

ADC番号は内部チャンネル番号に変換する。

```
1, 2, 4, 8, 16, 32 → 0, 1, 2, 3, 4, 5
```

サンプリング番号は1～50を0～49として保存する。ブロックの並び順に関係なく、`adc[チャンネル, サンプル, データ項目]`へ整理される。

波形解析

- 最初の23サンプルの平均をペデスタルとする。
- ペデスタルを全波形から差し引く。
- LG4信号の合計が最大となるサンプルを`peak_id`とする。
- 通常は`peak_id - 4`～`peak_id + 9`を台形則で積分する。
- `-constpeak`指定時は、サンプル25～41を固定で積分する。
- HG/LGの4信号比から位置`x, y`を計算する。
- ADC値が`16383`に達した場合、飽和と判定する。
  - `satu_h`：HGの飽和
  - `satu_l`：LGの飽和

通常のHDF5出力

ヒットしたチャンネルごとに1行保存する。主なデータは以下の通り。

```
trig, event_time, ch, hitnum
hg, lg, x_hg, y_hg, x_lg, y_lg
wave_sum, ped, peak_id
satu_h, satu_l, temp, temp_raw
```

出力チャンネルは次式で変換される。

```
出力ch = 5 - 内部チャンネル番号
```

`-adc` 指定時

通常の `.h5` に加えて `_adc.h5` を作成し、イベントごとの未補正ADC波形を保存する。

```
adc[event, channel, sample, item]
```

形状は次の通り。

```
イベント数 × 6チャンネル × 50サンプル × 10項目
```

10項目は以下の通り。

```
[ADC番号, サンプル番号, HG0～HG3, LG0～LG3]
```

ペデスタルは同じファイルの `ped` に保存される。

実行例

```
python3 psa_decode.py -i data.dat
python3 psa_decode.py -i data.dat --adc
python3 psa_decode.py -i data.dat --constpeak
python3 psa_decode.py -i data.dat --retry
```

HDF5ファイルは通常 `/home/smile/data` に保存され、入力ファイルの場所にはシンボリックリンクが作成される。リンク先は `--link-dir` で変更できる。

バタフライマップから実座標への変換ファイル

ピークサーチをかけたのちに、正方形に補正し、ピクセルに割り当てている。

peak_search.py

`peak_search.py` は、 `psa_decode.py` のHDF5出力から各チャンネルの位置分布を作成し、最大64個のピークを検出するプログラムである。

検出したピーク位置はCSVに保存され、 `grid_spect.py` の位置補正に使用される。

```
.dat
 ↓ psa_decode.py
イベントHDF5
 ↓ peak_search.py
ピーク位置CSV
 ↓ grid_spect.py
8×8位置補正・スペクトル作成
```

入力

HDF5から以下を読み込む。

```
hg, x_hg, y_hg, ch
```

複数のHDF5がある場合は、すべてまとめて解析する。

イベント選別

デフォルトでは、次の範囲の `hg` だけを使用する。

```
30,000 <= hg <= 500,000
```

範囲は変更できる。

```
python3 peak_search.py --emin 50000 --emax 400000
```

現在は`peak_id`や飽和フラグによる選別は行わない。

ピーク検出

チャンネルごとに`x_hg`対`y_hg`の2次元ヒストグラムを作成し、次の処理を行う。

1. 位置分布を平滑化する
2. 周囲より大きい局所最大値を探す
3. 近すぎるピークを除外する
4. 強度順に最大64個を採用する

デフォルト設定は以下の通り。

```
ヒストグラム：
x：-0.32～0.32、250ビン
y：-0.16～0.16、300ビン

平滑化：各方向7ビン幅
最小ピーク間隔：3ビン
最大ピーク数：64
```

ピーク番号`No`は強度順であり、8×8ピクセル番号ではない。`grid_spect.py`が座標を並べ替えてピクセル番号を決める。

出力

デフォルトでは、以下を作成する。

```
peak_output/position_.csv
peak_output/peaks_map_.png
```

CSVの内容は以下。

```
ch,No,x,y,z
```

- `ch`：チャンネル番号
- `No`：検出順
- `x, y`：ピーク位置
- `z`：元ヒストグラムのカウント数

- `-show`を指定すると、ピークマップを画面にも表示する。

```
python3 peak_search.py --show
```

入力ディレクトリ

指定ディレクトリ直下にHDF5があれば、それらをまとめて処理する。直下にHDF5がなければ、サブディレクトリごとに処理する。

タグや出力先も指定できる。

```
python3 peak_search.py --tag run1
python3 peak_search.py -o results --tag run1
```

注意点

- 各チャンネルで64ピーク検出できない場合、`grid_spect.py`の補正は実行できない。
- `position_.csv`とピークマップで、実際の64ピークが正しく検出されているか確認する必要がある。
- 入力ディレクトリ内の全HDF5を読むため、不要な`_adc.h5`や別形式のHDF5があるとエラーになる可能性がある。
- 構文チェックは済んでいるが、実データでの検出結果は未確認である。

grid_spect.py

`grid_spect.py`は、`psa_decode.py`のHDF5出力を読み込み、位置補正を行って8×8ピクセルごとのHGスペクトルを作成する後段解析プログラムである。

```
.dat
 ↓ psa_decode.py
イベントHDF5
 ↓ grid_spect.py
位置補正・ピクセル分割
 ↓
ピクセルごとのHGスペクトル
```

入力

カレントディレクトリの`.h5`を読み込む。ただし、出力ファイル`gridcorr_events.h5`は除外する。

使用するデータは主に以下。

```
x_hg, y_hg, ch, hg, peak_id, satu_h, satu_l
```

位置補正には、各チャンネル64点のピーク位置を含む

```
peak_output/position_.csv
```

を使用する。

イベント選別

通常は次の条件を満たすイベントだけを使用する。

```
peak_id > 25
satu_h == 0
satu_l == 0
```

- `-all`を指定すると飽和イベントも含める。ただし、`peak_id > 25`は常に適用される。

```
python3 grid_spect.py
python3 grid_spect.py --all
```

位置補正とピクセル分割

CSVの64点を、まずx方向に8グループへ分け、各グループ内でy方向に並べて8×8の理想座標と対応付ける。

対応点を使い、チャンネルごとに`RBFInterpolator`（薄板スプライン）で位置を補正する。

```
(x_hg, y_hg) → (x_corrected, y_corrected)
```

補正後の座標を基に、各イベントをピクセル`(n, m)`へ割り当てる。判定範囲は半開区間である。

```
n - 0.5 <= x_corrected < n + 0.5
m - 0.5 <= y_corrected < m + 0.5
```

HGスペクトル

各チャンネル・各ピクセルについて、`hg`のヒストグラムを作成する。

```
範囲：0～400,000
ビン数：400
形状：8×8×400
```

保存配列は次の形式。

```
spectra_chN[n - 1, m - 1, :]
```

位置マップ

各チャンネルについて、補正前後の位置分布を作成する。

```
before：x_hg, y_hg
after ：x_corrected, y_corrected
```

補正後は、外挿による範囲外イベントも確認できるよう、表示範囲を`-2～11`とする。カウントは対数表示する。